

Dynamic Resource Allocation in Cloud Computing

via Deep Reinforcement Learning

EECS 6893

Project Proposal

Minimizing power consumption
while enforcing SLA constraints
through intelligent scheduling

Project Introduction

Cloud data centers consume an immense and growing share of global electricity. The way workloads are mapped to physical servers has a direct, measurable impact on energy consumption.

This project develops a **dynamic resource allocation scheduler** that decides in real time which server handles each job — optimizing for minimal power while guaranteeing SLA compliance.

MDP Formulation

State S_t : CPU/memory utilization, power states, job queue characteristics

Action A_t : Assign head-of-queue job to server $k \in \{1, 2, \dots, N\}$

Reward R_t : $R = -(\alpha \cdot \text{Power} + \beta \cdot \text{SLA violations})$

Data & Methodology

Simulation built using **Gymnasium**, driven by real workload traces from **Google Cluster Data** and **Alibaba Cluster Traces**

Agents implemented in PyTorch using DQN/PPO, benchmarked against standard heuristic schedulers.

Identifying the Problem

Traditional cloud schedulers rely on **static heuristics** like **Round Robin** (which cycles through servers blindly), **Shortest Job First** (which ignores server load dynamics), or **First-Fit Decreasing** (which packs greedily without long-term foresight). These approaches are fundamentally rigid.

Delayed & cascading consequences: A poor scheduling decision at time t may not immediately trigger an SLA violation, but it can silently degrade server headroom, causing a cascading bottleneck dozens or hundreds of steps later when a burst of jobs arrives and no server has sufficient capacity.

Power-SLA tradeoff is inherently adversarial: aggressively consolidating workloads onto fewer servers saves power but risks SLA breaches, while spreading workloads keeps latency low but wastes energy on idle servers.

Why Reinforcement Learning?

Static heuristics fail

Round Robin, SJF, and FFD cannot adapt to non-linear power dynamics or bursty workload patterns

No ground truth exists

Optimal scheduling over a large cluster is NP-hard, so supervised learning has no labels to learn from

Scales to high-dimensional state spaces

Unlike exact DP, deep RL approximates policies via neural networks without requiring a tractable analytical model of cluster dynamics

Natural sequential decision structure

Each scheduling action changes cluster state, affecting all future decisions — MDPs model this dependency natively

Proposed Approach 1

Agentic RL

Decision Transformer

Can treat RL as sequence modeling — condition on desired SLA return at inference to generate actions, without traditional value-function bootstrapping

SLA Optimization Agent

RL agent (with internal sub-agents) focused on meeting latency and throughput SLA targets — ensures jobs complete within acceptable bounds

Power Assignment Agent

RL agent (with internal sub-agents) dedicated to minimizing cluster power consumption — learns to consolidate workloads and manage server power states

Supervisor Module

Mediates the SLA vs. Power tradeoff at each timestep, balancing competing objectives across the two agents via learned weighting

Proposed Approach 2

Offline RL from Cluster Traces

Key Advantage

SLAs are enforced as hard constraints ($P(\text{violation}) \leq \epsilon$) with formal guarantees — not as soft penalties that can be gamed by the agent

Learning from Real Cluster Data

Scheduling policy learned directly from Google Cluster Data and Alibaba Cluster Traces — millions of real job submissions with resource requests, durations, and machine assignments

Offline RL Framework

No live environment interaction needed — the policy is trained entirely on historical data batches, avoiding the cost and risk of online exploration in a production cluster

Constrained MDP (CMDP)

Hard SLA constraints enforced via Lagrangian relaxation rather than reward shaping — completely separates the constraint from the objective, avoiding the fragile α/β balancing problem

Tentative Timeline

Weeks 1–2 Environment & Data

Build custom Gymnasium simulation environment with realistic power curves; parse Google/Alibaba cluster traces into standardized job queue format; define MDP state/action spaces

Weeks 3–4 Baselines & Architecture

Implement heuristic baselines (Round Robin, SJF, FFD); build multi-agent framework for Approach 1 and offline RL + CMDP pipeline for Approach 2 in PyTorch

Weeks 5–6 Training & Experiments

Train both approaches, tune hyperparameters and reward weights; run comparative experiments tracking power consumption, SLA violation rate, and latency

Week 7 Analysis & Report

Finalize visualizations (training curves, Pareto frontiers, utilization heatmaps), statistical analysis, and write the final project report